

PRACTICAL WEEK - 6

TASK 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    int fd_read, fd_write;

    // Create the FIFOs (named pipes)
    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started. Waiting for client...\n");

    // Open FIFO1 for reading (blocks until client opens for writing)
    fd_read = open(FIFO1, O_RDONLY);
    read(fd_read, message, sizeof(message));
    close(fd_read);

    printf("Server received: %s\n", message);

    // Open FIFO2 for writing
    fd_write = open(FIFO2, O_WRONLY);
    char response[] = "Message received by server.";
    write(fd_write, response, strlen(response) + 1);
    close(fd_write);

    printf("Server response sent.\n");

    // Clean up FIFOs
    unlink(FIFO1);
    unlink(FIFO2);

    return 0;
}
```

```
rishill10@ASUS16X: ~/Week6
$ mkdir Week6
$ cd Week6
$ nano server.c
$ gcc server.c -o server
$ ./server
Server started. Waiting for client...
```

```
rishill10@ASUS16X: ~/Week6 × + v
GNU nano 8.7.1 client.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main() {
    char message[100];
    char response[100];
    int fd_write, fd_read;

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);
    message[strcspn(message, "\n")] = '\0'; // Remove newline

    // Open FIFO1 for writing
    fd_write = open(FIFO1, O_WRONLY);
    write(fd_write, message, strlen(message) + 1);
    close(fd_write);

    // Open FIFO2 for reading
    fd_read = open(FIFO2, O_RDONLY);
    read(fd_read, response, sizeof(response));
    close(fd_read);

    printf("Server response: %s\n", response);

    return 0;
}
```

```
rishill10@ASUS16X: ~/Week6 × + v
rishill10@ASUS16X:~$ cd Week6
rishill10@ASUS16X:~/Week6$ gcc server.c -o server
gcc client.c -o client
rishill10@ASUS16X:~/Week6$ ./server
Server started. Waiting for client...
Server recived: hello server
server response sent
```

TASK 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
rishill10@ASUS16X: ~/Week6 × + v
GNU nano 8.7.1 p-6.c *
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler function
void signal_handler(int signum) {
    if (signum == SIGINT) {
        printf("SIGINT received (Ctrl+C).\n");
    } else if (signum == SIGUSR1) {
        printf("SIGUSR1 received.\n");
    } else if (signum == SIGTERM) {
        printf("SIGTERM received.\n");
        exit(0); // Gracefully exit on SIGTERM
    }
}

int main() {
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGUSR1, signal_handler);
    signal(SIGTERM, signal_handler);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    // Infinite loop waiting for signals
    while (1) {
        pause(); // Suspends process until a signal is received
    }

    return 0;
}
```



rishill10@ASUS16X: ~/Week6



Ubuntu



```
rishill10@ASUS16X:~$ cd Week6
rishill10@ASUS16X:~/Week6$ nano p-6.c
rishill10@ASUS16X:~/Week6$ ./p62
-bash: ./p62: No such file or directory
rishill10@ASUS16X:~/Week6$ gcc p-6.c -o p62
rishill10@ASUS16X:~/Week6$ ls
client      client_to_server  p62          server.c
client.c    p-6.c            server       server_to_client
rishill10@ASUS16X:~/Week6$ ./p62
Signal handling program started.
PID: 1471
Waiting for signals...
^CSIGINT received (Ctrl+C).
SIGUSR1 received.
SIGTERM received.
rishill10@ASUS16X:~/Week6$ |
```